

Mini Project 1 — Azure Cloud Services

Hosting the Studio Ghibli API in the Cloud

HES-SO Valais-Wallis — 64-41 Cloud Technologies · Lorenzo Forte & Marion Sarrasin · Teacher: Alain Duc · 23 April 2026

1. Introduction

Studio Ghibli is a renowned Japanese animation studio whose films — among them *My Neighbor Totoro*, *Spirited Away* and *Princess Mononoke* — have earned a worldwide fanbase. To serve this community, the developer Janai Pakos built a lightweight REST API cataloguing the studio's films, people, locations, species and vehicles.

Since December 1, 2022 the original Heroku-hosted version of the API is no longer free, because Heroku discontinued its free plans. The source code, however, has been preserved on GitHub, so anyone can re-publish it on the cloud platform of their choice. The goal of this mini project is exactly that: to host the Ghibli API on Microsoft Azure so that fans can reach it again from any browser or HTTP client such as Postman. This document describes the technical requirements of the application, compares the hosting options that were considered, justifies the chosen solution, details the deployment, and provides the testing URL and the evaluator access.

2. Technical Requirements

The Ghibli API is a small Node.js application built around `json-server`, which turns a static JSON file into a fully-featured REST API. Understanding its needs is the prerequisite to choosing the right Azure service.

- **Runtime and package manager:** Node.js with npm. The original `package.json` declares the engine as Node 7.x, which is long deprecated; we use Node 22 LTS, on which `json-server` remains compatible.
- **Dependencies:** `json-server` (REST endpoints from a JSON file) and `clone`, installed via `npm install`.
- **Entry point & data:** the application starts with `npm start` (runs `node index`). All data is served from a single static `data.json` file — no external database is required, making the app fully stateless.
- **Networking:** the service must be publicly reachable over HTTPS. It binds to the port provided by the `PORT` environment variable, the standard convention used by PaaS providers.
- **Resource footprint:** a single Node process serving a few hundred kilobytes of JSON from memory — a minimal CPU/memory tier is enough.
- **Accounts & tooling:** an Azure subscription (Azure for Students in our case), a GitHub account to host the source and run CI/CD, and any modern IDE (JetBrains Rider, VS Code, etc.) to inspect or edit the code.

3. Different Solutions Considered

Microsoft Azure offers several services that can host a small Node.js application. Four options were evaluated against the requirements above.

3.1 Azure App Service (PaaS)

A fully managed Platform-as-a-Service for web applications. It natively supports Node.js, provides HTTPS on an `*.azurewebsites.net` domain out of the box, integrates directly with GitHub for continuous deployment, and offers a Free (F1) and Basic (B1) tier well suited to a demo API. Downsides are limited flexibility compared to a raw VM and cold starts on the free tier.

3.2 Azure Virtual Machine (IaaS)

A Linux VM (for example Ubuntu B1s) on which Node.js, the repository and a process manager such as PM2 would be installed manually, typically behind an NGINX reverse proxy. A process manager is a supervisor tool that keeps the Node application running in the background, restarts it automatically on crash, and relaunches it when the server reboots — a role that App Service fulfils out of the box. This option gives maximum control but adds significant operational overhead (OS patching, TLS, monitoring) and costs more than a shared PaaS tier for such a small workload.

3.3 Azure Container Instances / Container Apps

The API could be packaged as a Docker image and run on Azure Container Instances (single container) or Container Apps (serverless, scale-to-zero). These options are portable and reproducible, but require writing and maintaining a Dockerfile and a container registry workflow — unnecessary complexity for a codebase that already works with a Node buildpack.

3.4 Azure Static Web Apps + Azure Functions

Serving the JSON data through HTTP-triggered Azure Functions behind a Static Web App was also considered. It would be very cheap, but the original code is an Express/json-server application, not a set of serverless functions. Porting it would require a non-trivial refactor and would defeat the purpose of reusing the archived code as-is.

Solution	Effort	Cost	Git CI/CD	Fit for this project
App Service (PaaS)	Very low	Free tier	Native	Excellent
Virtual Machine (IaaS)	High	Higher	Manual	Overkill
Container Apps / ACI	Medium	Low	Via registry	Unnecessary complexity
Static Web Apps + Functions	High (rewrite)	Very low	Native	Requires refactoring

4. Final Decision and Why

Our choice is Azure App Service (Linux, Node.js 22 LTS runtime), deployed continuously from GitHub via GitHub Actions. It offers the best trade-off between simplicity, cost and fitness for this workload:

- **Simplicity:** the Ghibli API is a small, stateless Node.js app — exactly the type of workload App Service is designed for. No Dockerfile, no VM, no serverless rewrite.
- **Cost:** the Free F1 tier (shared infrastructure, 1 GB memory) is more than enough for a demo API and fits perfectly within the Azure for Students subscription.
- **Deployment speed:** native GitHub integration — every push to the master branch triggers an automatic redeployment through GitHub Actions.
- **Low operational burden:** HTTPS, public domain, OS patching, process supervision (auto-restart on crash) and runtime updates are all handled by Azure.

5. Solution Deployment

The deployment was carried out entirely through the Azure Portal and followed the steps below.

5.1 Forking the source repository

The archived repository janaipakos/ghibliapi was forked to ForteLorenzo/ghibliapi so that App Service could connect to a repository owned by the students, and so that Azure could push the generated GitHub Actions workflow file into it.

To align with the .Node version, modify the package.json file engine key, to :

```
"engines":{  
  "node":"~22"  
},
```

5.2 Creating the Azure resources

- **Resource Group:** ghibli-api-hevs_group in the Switzerland North region (Azure for Students subscription).
- **Web App:** ghibli-api-hevs — publish *Code* (not Container), OS Linux, runtime Node 22 LTS. The default short hostname was kept (ghibli-api-hevs.azurewebsites.net) — Azure’s optional *Secure unique default hostname* feature was not enabled in order to keep a clean testing URL.
- **App Service Plan:** ASP-ghibliapihevsrgroup-90fb, Linux, Free F1 tier. No database, Application Insights or Defender was enabled, so the whole project stays within the Free tier.

5.3 Configuring continuous deployment

Continuous deployment was configured after creation through the Deployment Center blade, because the F1 Free tier does not allow GitHub Actions setup during Web App creation. Source was set to GitHub, organisation ForteLorenzo, repository ghibliapi, branch master, with the option “Add a workflow” so that Azure commits a workflow file directly in the repo. We chose the modern “User-assigned managed identity” authentication (OIDC): Azure creates a federated credential between Microsoft Entra ID and GitHub, so the workflow can deploy without storing a long-lived publish-profile secret. Azure then generated the workflow file master_ghibli-api-hevs.yml and GitHub Actions triggered the first deployment automatically, running npm install, the optional build step, and publishing the artifact to the Web App.

5.4 Verification

Once the GitHub Actions pipeline completed successfully, the Web App was tested both in the browser and in Postman. All documented endpoints (/films, /people, /locations, /species, /vehicles) returned the expected JSON payloads, including resource-by-id requests such as /films/{id}.

6. URL for Testing

The public base URL of the deployed API is:

<https://ghibli-api-hevs.azurewebsites.net>

Example requests that can be tested in a browser or in Postman:

- GET <https://ghibli-api-hevs.azurewebsites.net/films>
- GET <https://ghibli-api-hevs.azurewebsites.net/films/58611129-2dbc-4a81-a72f-77ddfc1b1b49> — My Neighbor Totoro (canonical sample from the original README)
- GET <https://ghibli-api-hevs.azurewebsites.net/people> · [/locations](https://ghibli-api-hevs.azurewebsites.net/locations) · [/species](https://ghibli-api-hevs.azurewebsites.net/species) · [/vehicles](https://ghibli-api-hevs.azurewebsites.net/vehicles)

7. Access to Resources

The teacher has been granted the Reader RBAC role at the Resource Group level, which allows inspection of every resource (Web App, App Service Plan, deployment logs, GitHub Actions runs) without modification rights.

Resource Group: [Azure Portal link](#)

Web App: [Azure Portal link](#)

Source & CI/CD history: github.com/ForteLorenzo/ghibliapi · [Actions](#)

8. Conclusion

Re-hosting the Studio Ghibli API illustrates how managed PaaS offerings can bring an archived open-source project back to life with minimal effort. Starting from a clear understanding of the application's needs — a small, stateless Node.js app with no database — we compared four Azure options and chose App Service because it combines a Free tier, native GitHub Actions integration with OIDC authentication, automatic HTTPS and essentially no operational overhead. The final outcome is a working public REST API, continuously deployed from GitHub to a Node 22 LTS App Service in Switzerland North, serving all five Ghibli resource categories. Beyond this specific exercise, the approach we followed — map requirements, compare managed services, pick the simplest fit — is the same reasoning that applies to most real-world cloud migration decisions.

9. Uses of AI

Anthropic's Claude was used as a support tool during this project. The assistant helped structure the document, cross-checked the technical requirements extracted from the source repository against current Azure documentation, drafted the comparison of hosting options, clarified concepts such as process managers, the difference between GitHub-native and external Git sources in App Service, and the role of OIDC federated credentials, and proof-read the final English text. All technical decisions (choice of service, tier, region, resource names, authentication mode) and the actual deployment on Azure were carried out and verified by the students. AI output was treated as a draft that was read, edited and validated before being incorporated into this document.

10. References

- Studio Ghibli — Wikipedia: en.wikipedia.org/wiki/Studio_Ghibli
- Original API: github.com/janaipakos/ghibliapi · fork used: github.com/ForteLorenzo/ghibliapi
- Azure App Service documentation: learn.microsoft.com/azure/app-service
- Deploy Node.js on App Service (quickstart): learn.microsoft.com/azure/app-service/quickstart-nodejs
- GitHub Actions deployment with OpenID Connect: learn.microsoft.com/azure/app-service/deploy-github-actions
- Assigning Azure RBAC roles: learn.microsoft.com/azure/role-based-access-control/role-assignments-portal
- json-server: github.com/typicode/json-server · Postman: postman.com